



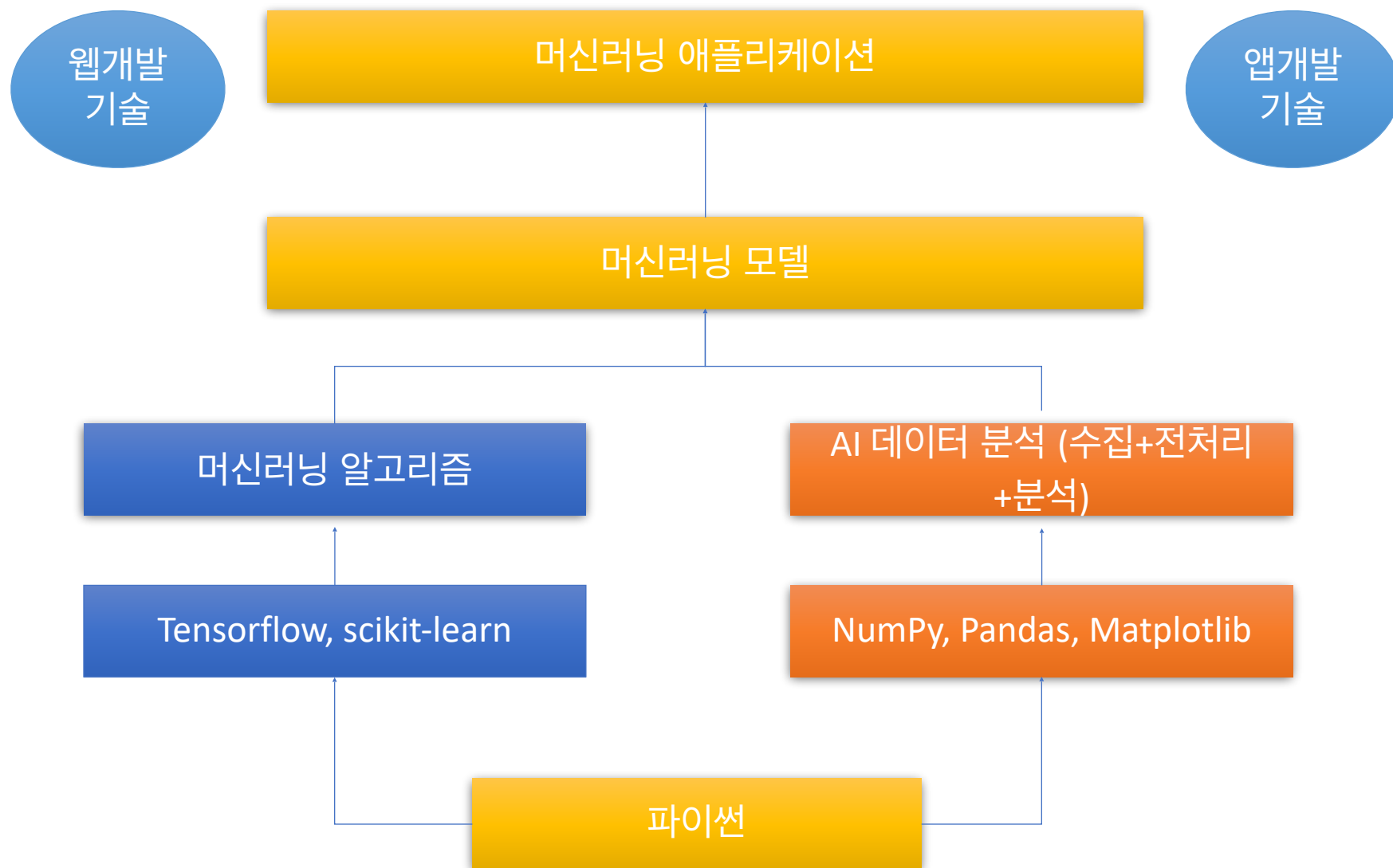
AI데이터분석

Contents

- **AI데이터분석 기초 프로그래밍**
 - 파이썬 프로그래밍 기초
 - 절차지향 프로그래밍
 - 파이썬 자료구조
 - 객체지향 프로그래밍
- **AI데이터분석 라이브러리**
 - 텍스트 데이터 처리
 - 넘파이 배열
 - 맷플롯립
 - 판다스
- **AI데이터분석 응용**
 - 데이터 수집
 - 데이터 전처리
 - 데이터분석과 머신러닝

3. AI데이터분석 응용

3-1. 데이터 수집





- 정형데이터(Structure data)
 - 미리 정해진 형식으로 구조화된 데이터
 - 예) 엑셀 시트, RDBMS 테이블
- 반정형데이터(Semi-structure data)
 - 특정한 형식에 따라 저장된 데이터이지만 정형데이터와 달리 형식에 대한 설명을 함께 제공
 - 구조를 해석하는 파싱(일종의 번역) 과정이 필요하며 파일 형태로 저장
 - 예) XML, JSON
- 비정형데이터(Unstructured data)
 - 정해진 구조가 없이 저장된 데이터
 - 빅데이터의 대부분을 차지
 - 예) 텍스트, 영상, 이미지 SNS

데이터 수집

데이터 전처리

데이터 분석/응용

- **데이터 수집**

- 오픈데이터 API, 웹 크롤링, 파일 읽기, DB 액세스

- **데이터 전처리 : 넘파이, 판다스 활용**

- 데이터 클린징, 데이터 연결과 병합, 데이터 재구조화

- **데이터 분석/응용**

- 데이터 시각화(맷플롯립 활용), 기계학습

- 데이터가 존재하는 곳

1. 미디어(SNS등)
2. 클라우드
3. 웹
4. 사물인터넷
5. 데이터베이스
6. 오픈 데이터/ API



출처 : 빅데이터분석과 머신러닝 (생능출판사)

- 판다스는 다양한 형식의 파일로부터 데이터 불러오기 함수 제공
 - 외부 데이터를 불러와 정형 데이터 구조(데이터 프레임)으로 변환
- 판다스에서 지원하는 외부 데이터 파일 형식
 - csv 파일
 - pd.read_csv() 함수
 - df.to_csv() 메소드
 - excel 파일
 - pd.read_excel() 함수
 - df.to_excel() 메소드
 - json 파일
 - pd.read_json() 함수
 - df.to_json() 메소드
 - 피클(pickle) 파일
 - pd.read_pickle() 함수
 - df.to_pickle() 메소드

- CSV(Comma-separated values)
- 각 라인의 컬럼들이 콤마로 분리된 텍스트 파일 포맷
- CSV 파일 읽기/저장
 - `df = pd.read_csv('파일경로/이름')`
 - `df.to_csv('파일경로/이름', index=False)`

```
import pandas as pd
file_path1= 'weather.csv'

df1= pd.read_csv(file_path1) #read_csv()함수로 데이터프레임 변환
df1.to_csv('sample_data.csv', index=False)

file_path2= 'sample_data.csv'
df2 = pd.read_csv(file_path2) #저장 파일 확인
print(df2)
```

	일시	평균기온	최대풍속	평균풍속
0	2010-08-01	28.7	8.3	3.4
1	2010-08-02	25.2	8.7	3.8
2	2010-08-03	22.1	6.3	2.9
3	2010-08-04	25.3	6.6	4.2
4	2010-08-05	27.2	9.1	5.6
...	...	...	...	...
3648	2020-07-27	22.1	4.2	1.7

- CSV 파일 읽기/저장

- `df = pd.read_excel('파일경로/이름')`
- `df.to_excel('파일경로/이름', index=False)`

```
[4] df1= pd.read_csv('weather.csv')  
df1.to_excel('weather.xlsx', index=False)
```

```
[6] import pandas as pd  
file_path1= 'weather.xlsx'  
  
df1= pd.read_excel(file_path1) #read_excel()함수로 데이터프레임 변환  
df1.to_excel('sample_data.xlsx', index=False)  
  
file_path2= 'sample_data.xlsx'  
df2 = pd.read_excel(file_path2) #저장 파일 확인  
print(df2)
```

	일시	평균기온	최대풍속	평균풍속
0	2010-08-01	28.7	8.3	3.4
1	2010-08-02	25.2	8.7	3.8
2	2010-08-03	22.1	6.3	2.9

- JavaScript Object Notation의 약자
- json 파일 읽기/저장
 - `df = pd.read_json('파일경로/이름')`
 - `df.to_json('파일경로/이름')`

```
[7] df1= pd.read_csv('weather.csv')  
df1.to_json('weather.json')
```

```
[10] import pandas as pd  
file_path1= 'weather.json'  
  
df1= pd.read_json(file_path1) #read_json()함수로 데이터프레임 변환  
df1.to_json('sample_data.json')  
  
file_path2= 'sample_data.json'  
df2 = pd.read_json(file_path2) #저장 파일 확인  
print(df2)
```

	일시	평균기온	최대풍속	평균풍속
0	2010-08-01	28.7	8.3	3.4
1	2010-08-02	25.2	8.7	3.8
2	2010-08-03	22.1	6.3	2.9

- 파이썬에서 객체를 저장하기 위한 이진 파일 형식
- pickle 파일 읽기/저장
 - `df = pd.read_pickle('파일경로/이름')`
 - `df.to_pickle('파일경로/이름')`

```
[11] df1= pd.read_csv('weather.csv')
      df1.to_pickle('weather.pkl')
```

```
[14] import pandas as pd
      file_path1= 'weather.pkl'

      df1= pd.read_pickle(file_path1) #read_pickle()함수로 데이터프레임 변환
      df1.to_pickle('sample_data.pkl')

      file_path2= 'sample_data.pkl'
      df2 = pd.read_pickle(file_path2) #저장 파일 확인
      print(df2)
```

	일시	평균기온	최대풍속	평균풍속
0	2010-08-01	28.7	8.3	3.4
1	2010-08-02	25.2	8.7	3.8
2	2010-08-03	22.1	6.3	2.9

- 웹 페이지를 원본 그대로 불러와 웹 페이지 내에 데이터를 추출하는 기술
 - 웹 스크래이핑(Scraping)이라고도 함
 - 크롤링을 전용 소프트웨어는 웹크롤러(Crawler)라고 함
 - 파이썬 크롤링 라이브러리 : BeautifulSoup
- BeautifulSoup을 이용한 웹 크롤링 절차
 - 데이터를 웹 크롤링하기 위해서는 웹소켓을 이용하여 원하는 웹사이트에 연결 요청을 진행
 - 연결 요청을 응답으로 웹서버는 응답(Response)를 보내면 보통 HTML이나 JSON 형식으로 반환
 - 반환된 HTML, JSON데이터를 Beautiful Soup으로 파싱

- 라이브러리 설치
 - 아나콘다 프롬프트에서 별도 설치
 - > !pip install BeautifulSoup4
- 라이브러리 포함
 - from bs4 import BeautifulSoup
 - from bs4 import BeautifulSoup as bs
 - bs 별칭 사용
- HTML문서 요청 라이브러리
 - requests 모듈 → 설치 필요
 - > !pip install requests
 - import requests
 - urllib.request 모듈
 - from urllib.request import urlopen

- requests 모듈 사용
 - `page = requests.get("https://www.daangn.com/hot_articles")`
 - `soup = BeautifulSoup(page.content, 'html.parser')`
 - 첫번째 인자 : HTML 텍스트(`page.content`, `page.text`)
 - 두번째 인자 : 파서 종류
 - `html.parser` : 기본 설치, 보통 속도
 - `lxml` : lxml 모듈 추가 설치 필요, 매우 빠름
 - `html5lib` : 브라우저와 같은 방식, 매우 느림
- urllib.request 모듈 사용
 - `fr = urlopen('https://www.naver.com/')`
 - `soup = BeautifulSoup(fr.read(), 'html.parser')`

- HTML 태그(요소) 이름을 속성으로 가짐
 - 태그(요소) 탐색 가능
 - ex) `soup.h2` : 첫번째 h2 태그 추출(태그+내용)
 - ex) `soup.h2.string == soup.h2.text` : 첫번째 p 태그의 내용만 추출



```
import requests
```

```
page = requests.get("https://www.daangn.com/hot_articles")  
soup = BeautifulSoup(page.content, "html.parser")
```

```
print(soup.h2)  
print('-----')  
print(soup.h2.string)  
print(soup.h2.text)
```



```
<h2 class="card-title">스피드랙 5단 첼제 선반 (11/15일 내려드려요)</h2>
```

```
스피드랙 5단 첼제 선반 (11/15일 내려드려요)  
스피드랙 5단 첼제 선반 (11/15일 내려드려요)
```


- `soup.find('태그명')`
 - 첫번째 태그명 추출 == `soup.태그명`
- `soup.find('태그명', attrs={'속성': '값'})`
 - `attrs` 속성을 가진 첫번째 태그 추출
 - cf) 태그의 내용만 추출 : `tag.text` 속성 == `tag.get_text()` 메소드

```
import requests
page = requests.get("https://www.daangn.com/hot_articles")
soup = BeautifulSoup(page.content, "html.parser")

print(soup.h2)
print(soup.find('h2'))
print(soup.find('h2').text)
print('-----')
print(soup.find('h1', {'id': 'hot-articles-head-title'}))

<h2 class="card-title">스피드랙 5단 첼제 선반 (11/15일 내려드려요)</h2>
<h2 class="card-title">스피드랙 5단 첼제 선반 (11/15일 내려드려요)</h2>
스피드랙 5단 첼제 선반 (11/15일 내려드려요)
-----
<h1 class="head-title" id="hot-articles-head-title">

    중고거래 인기매물
</h1>
```

- `soup.find_all('태그명')`
 - 태그명을 가진 모든 태그 추출
 - 반환타입 : `bs4.element.ResultSet`
- `soup.find_all('태그명', attrs={'속성': '값'})`
 - 태그명 중에서 `attrs` 속성을 가진 태그 추출



```
import requests
page = requests.get("https://www.daangn.com/hot_articles")
soup = BeautifulSoup(page.content, "html.parser")

print(soup.find_all("h2"))
print(len(soup.find_all("h2")))
print('-----')
print(soup.find_all(attrs={'class': 'head-title'}))
print(len(soup.find_all(attrs={'class': 'head-title'})))
```



```
[<h2 class="card-title">스피드랙 5단 첼제 선반 (11/15일 내려드려요)</h2>, <h2 class="card-title">위닉 82
```

```
-----
[<h1 class="head-title" id="hot-articles-head-title">
```

```
    중고거래 인기매물
</h1>]
```

- CSS 선택자를 통해 태그 추출

- `soup.select('.class명')`
 - class명을 가진 모든 태그 추출
- `soup.select('#id명')`
 - id명을 가진 태그 추출

- cf) 반환타입 : `bs4.element.ResultSet` → 원소 : `bs4.element.Tag`

```
[17] import requests

page = requests.get("https://www.daangn.com/hot_articles")
soup = BeautifulSoup(page.content, "html.parser")

for x in range(0,10):
    print(soup.select(".card-title")[x].get_text())
```



스피드랙 5단 첼제 선반 (11/15일 내려드려요)
위닉스 제습기
광주 김치축제 배추김치 판매 (10kg)
4구 20m 캠핑릴선 (usb 충전가능)
한국시리즈 티켓
텐트
삼성무풍에어컨+벽걸이+실외기
LG 코드제로 A9 5만원
18k 3돈 금팔찌 팔아요
노스페이스 눅시

- bugs 사이트에서 음악 순위 크롤링

```
▶ from urllib.request import urlopen
   from bs4 import BeautifulSoup

   fr = urlopen("https://music.bugs.co.kr/chart?wl_ref=M_left_02_01")
   soup = BeautifulSoup(fr.read(), 'html.parser')

   # td 태그에 check라는 class가 있는 td 태그를 모두 가져온다.
   musics = soup.find_all('td', "check")

   # musics의 각 태그들에 대해서
   for i, music in enumerate(musics):
       # input 태그안에 title 속성값을 parsing한다.
       print("{}위: {}".format(i+1, music.input['title']))
```

```
➞ 1위: Perfect Night
   2위: Baddie
   3위: You & Me
   4위: Love Lee
   5위: Get A Guitar
   6위: 후라이의 꿈
   7위: 잠시라도 우리
```



thank you

본 과제(결과물)는 교육부와 한국연구재단의 재원으로 지원을 받아 수행된
디지털신기술인재양성 혁신공유대학사업의 연구결과입니다.